

Artificial Intelligence

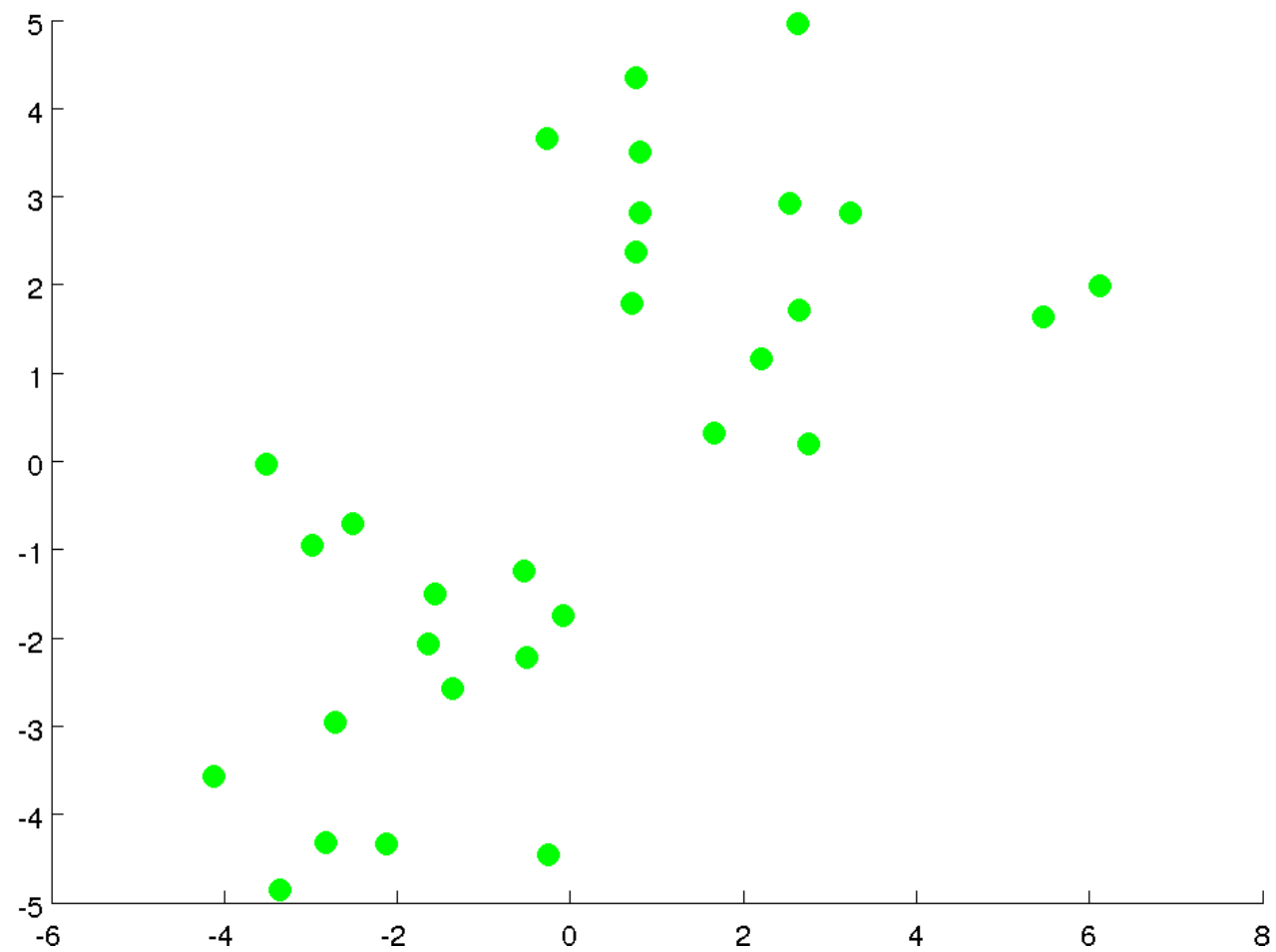
AI2002

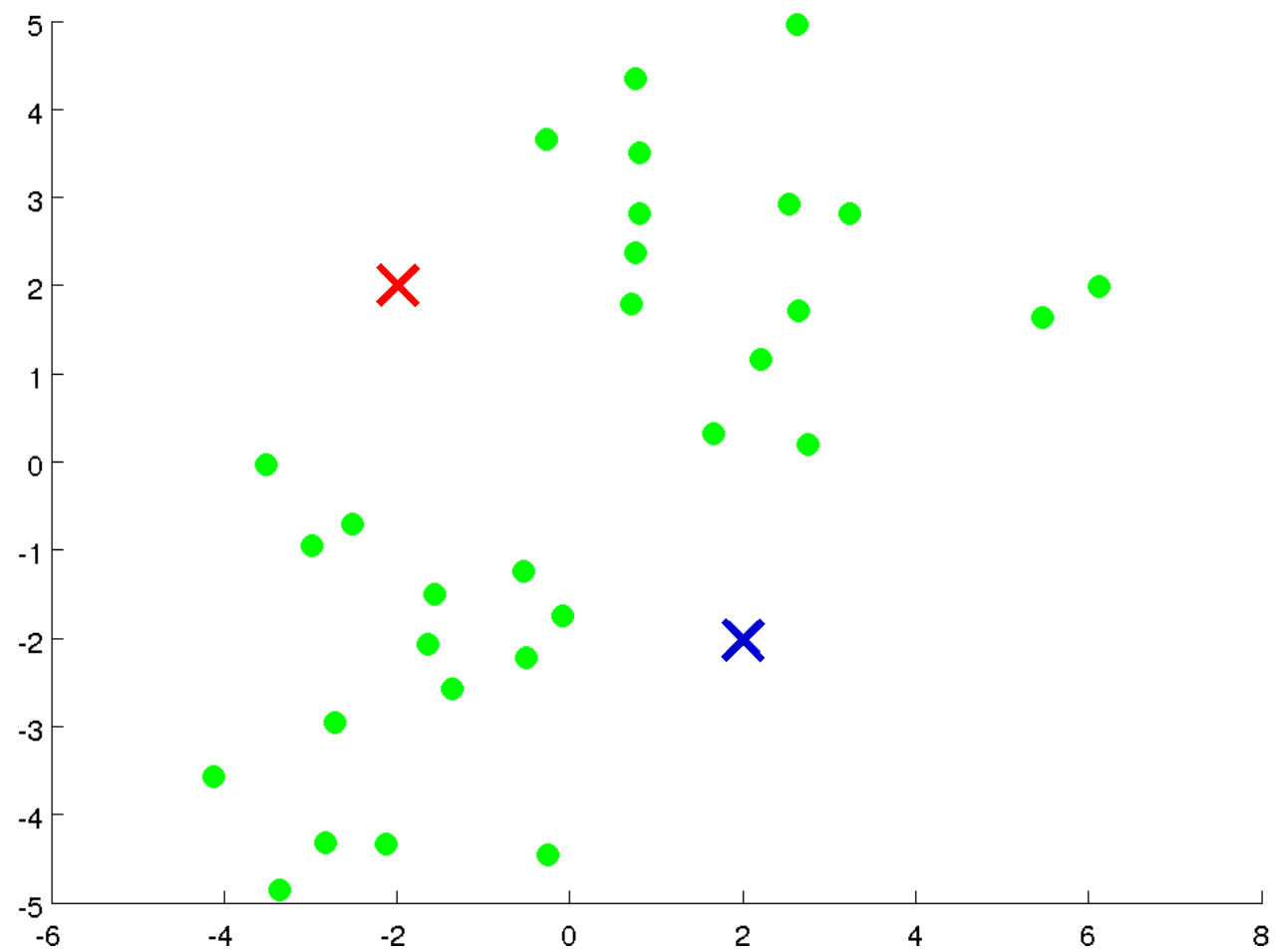
Rushda Muneer

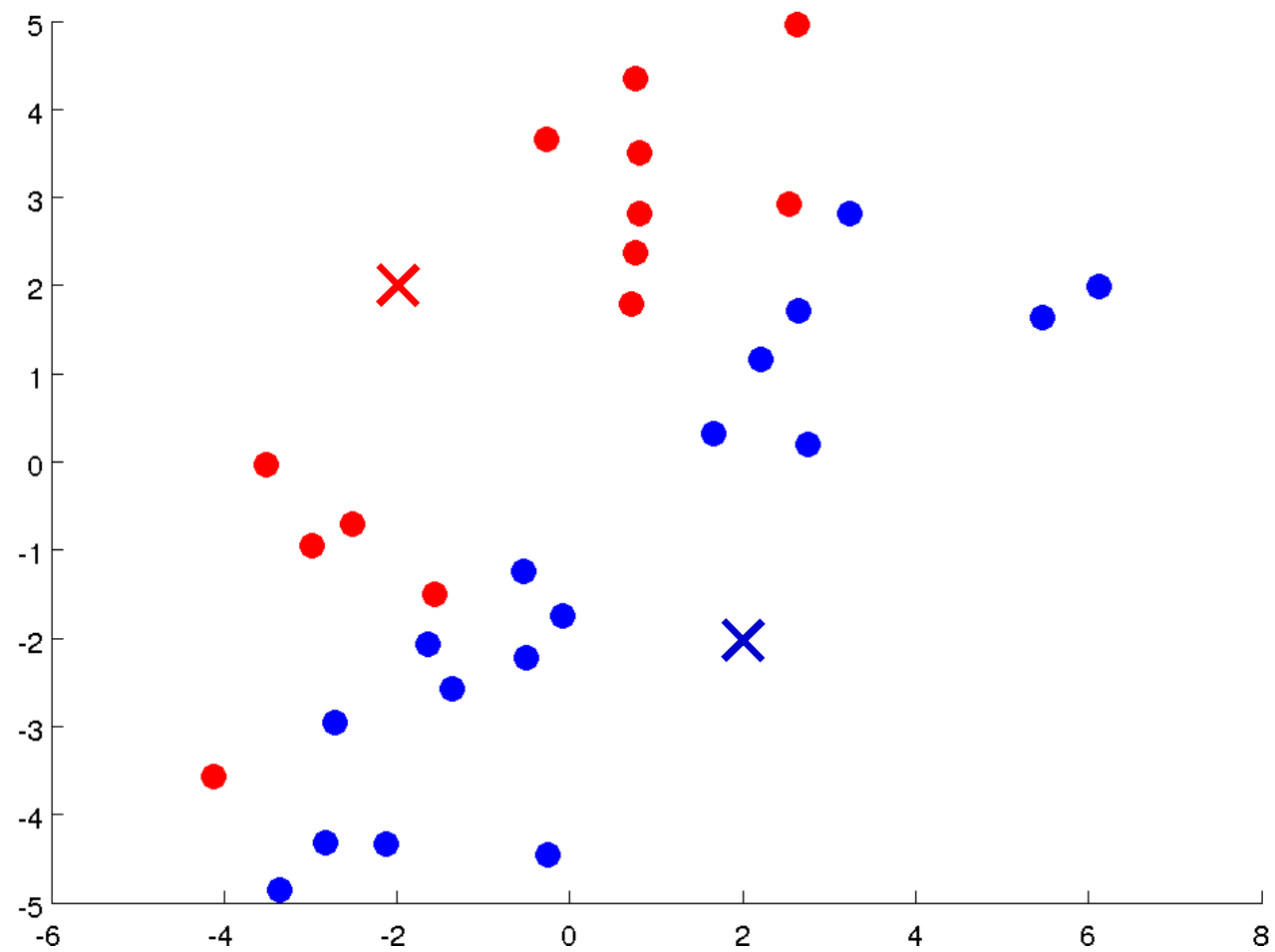
Spring 2026

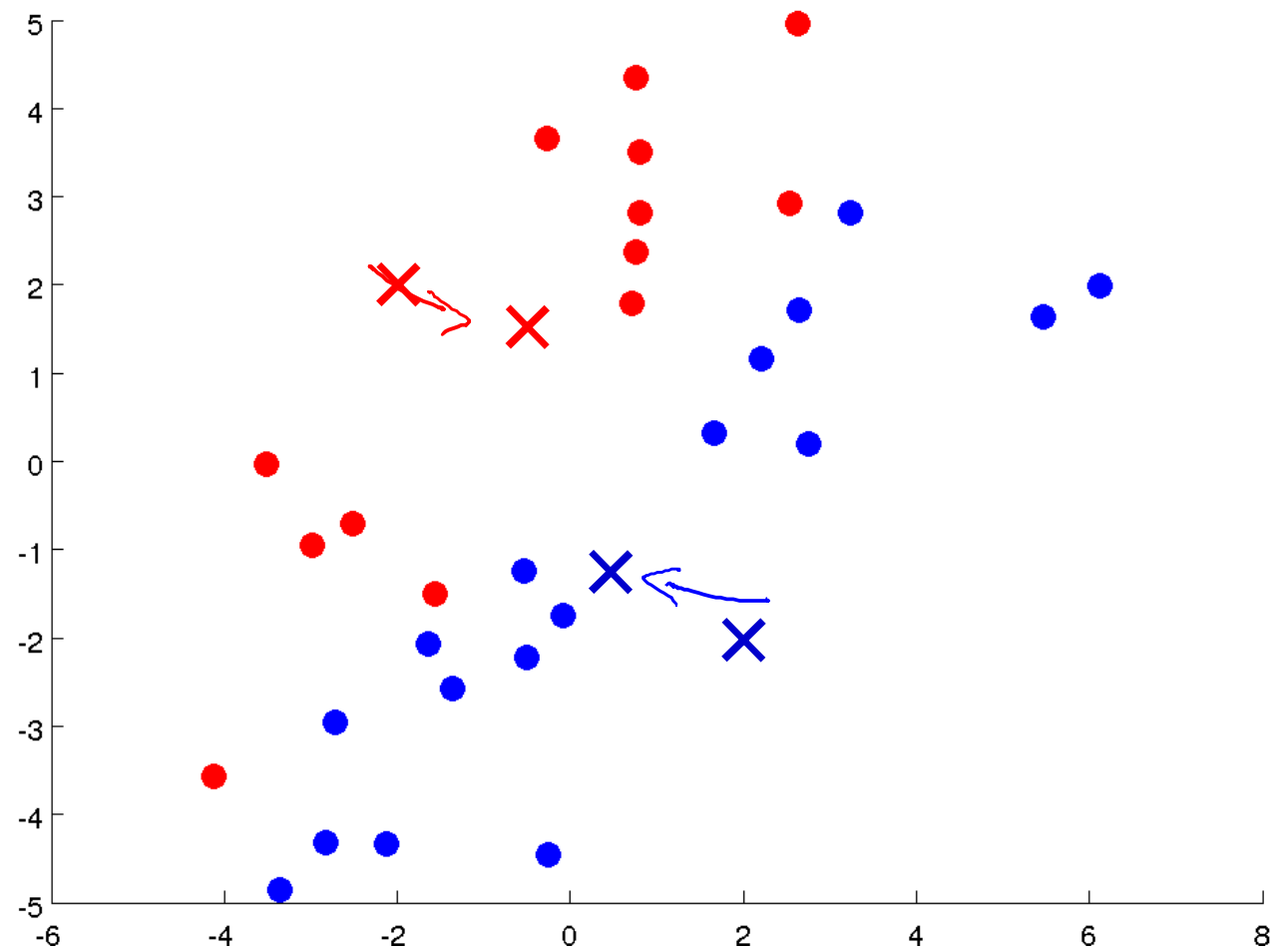
Clustering

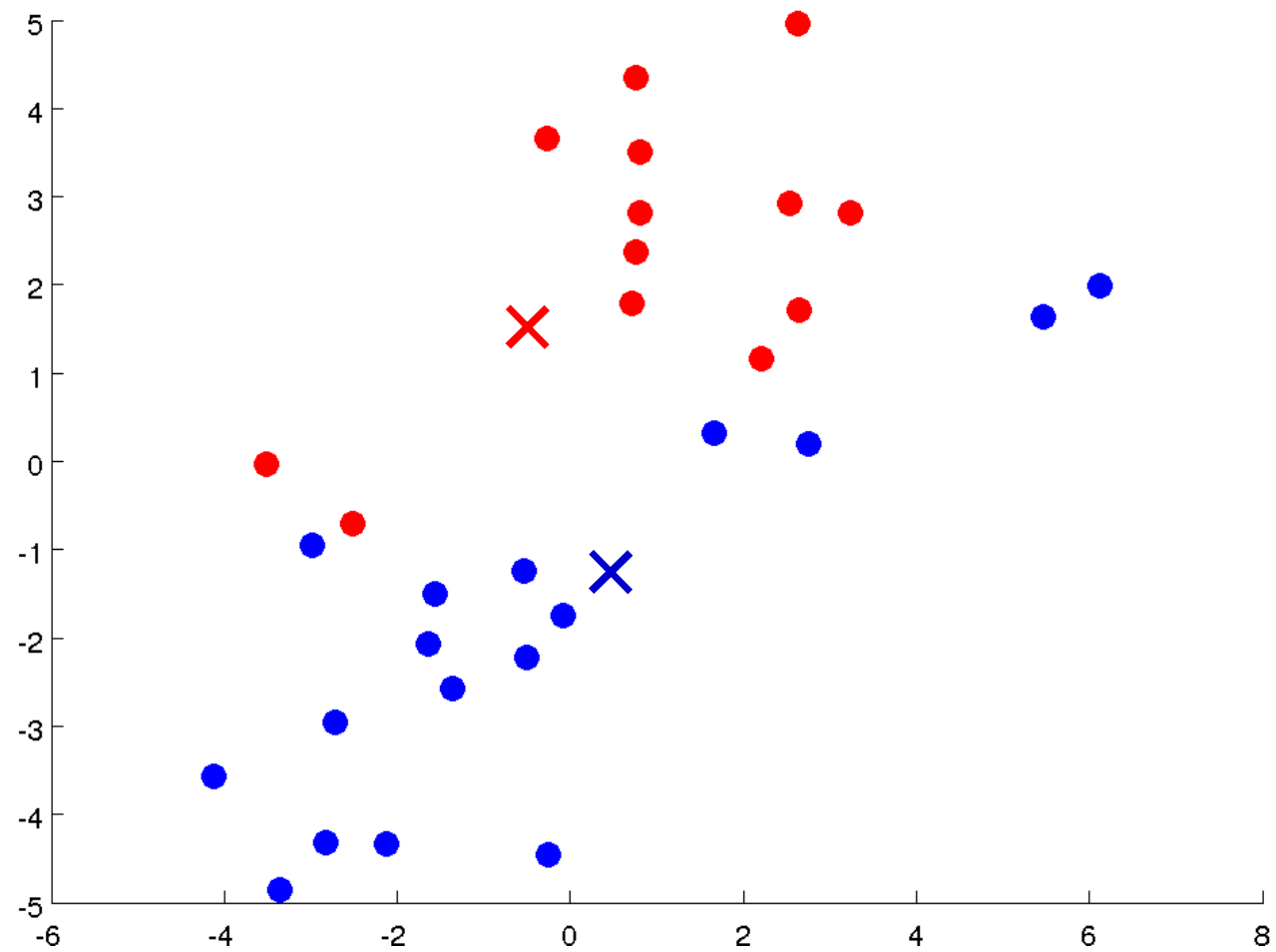
K-means Algorithm

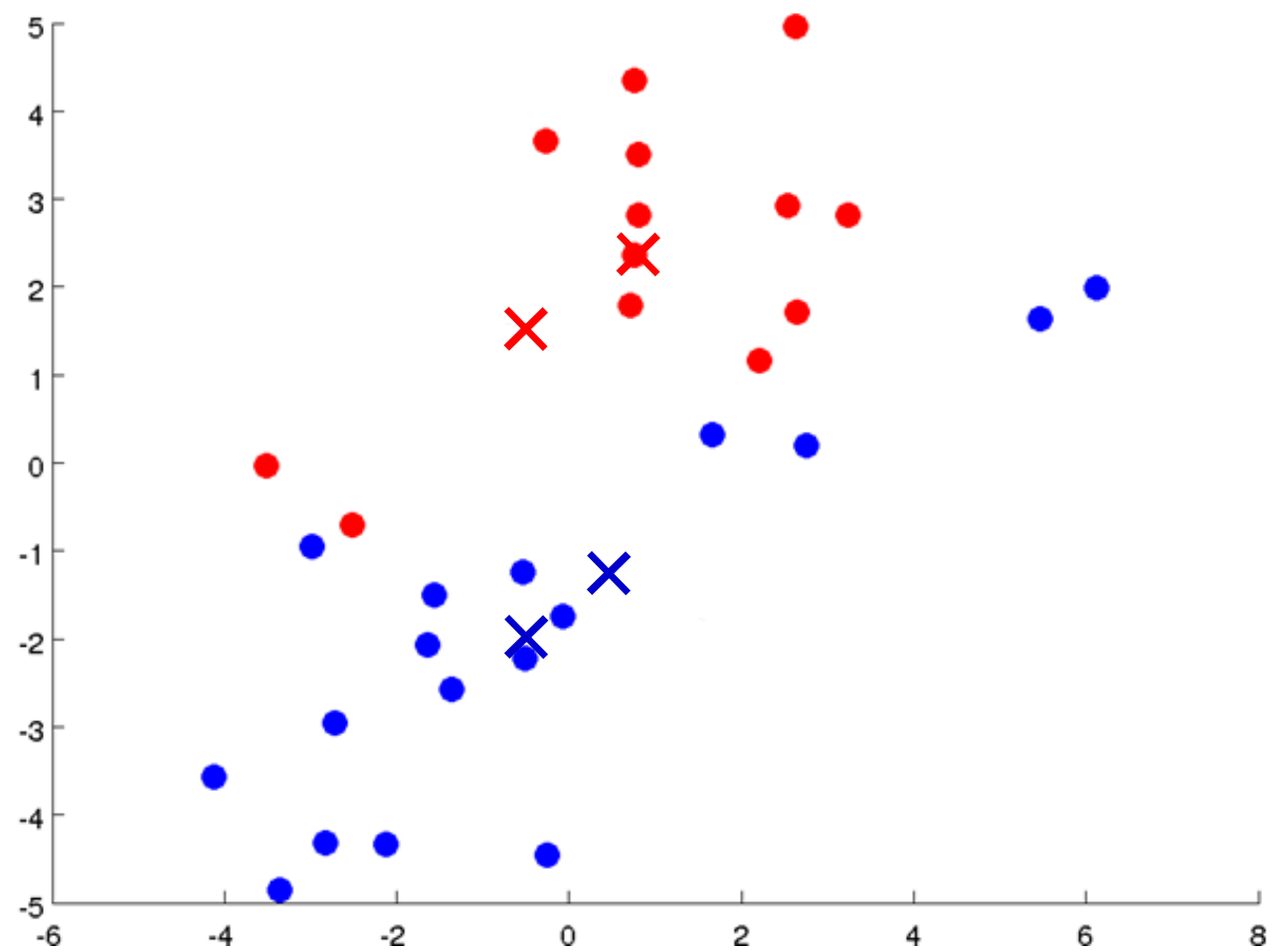


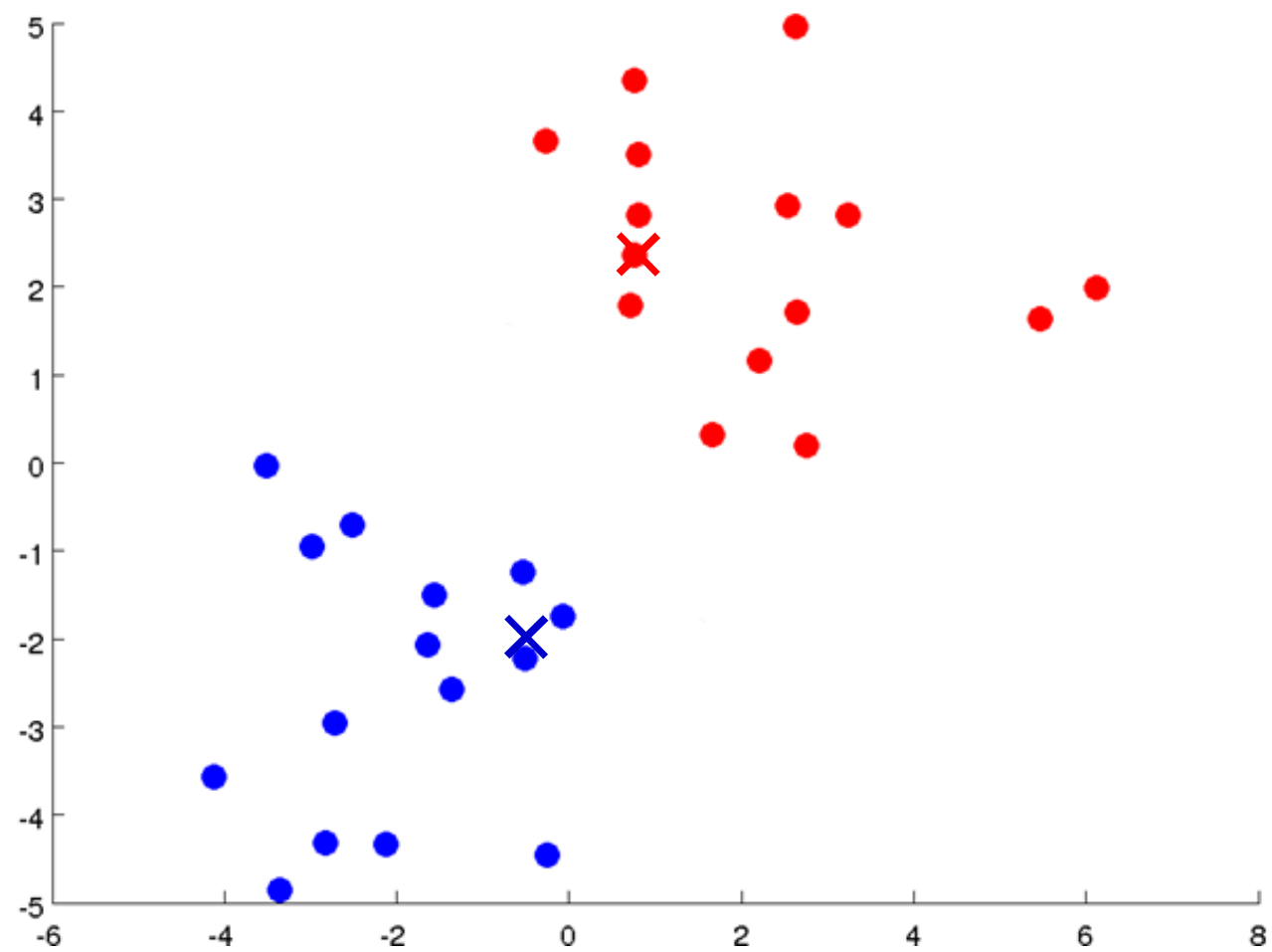


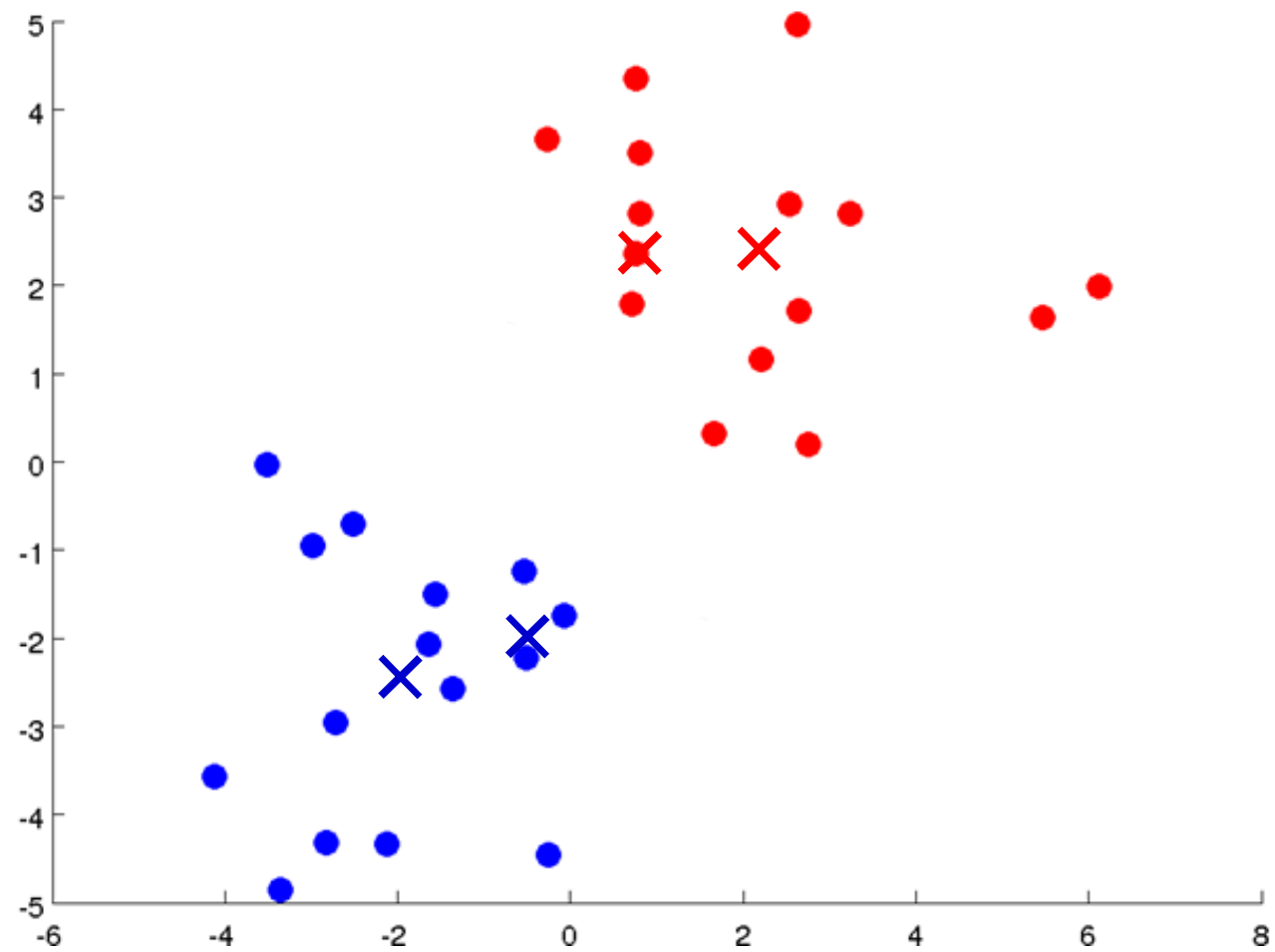


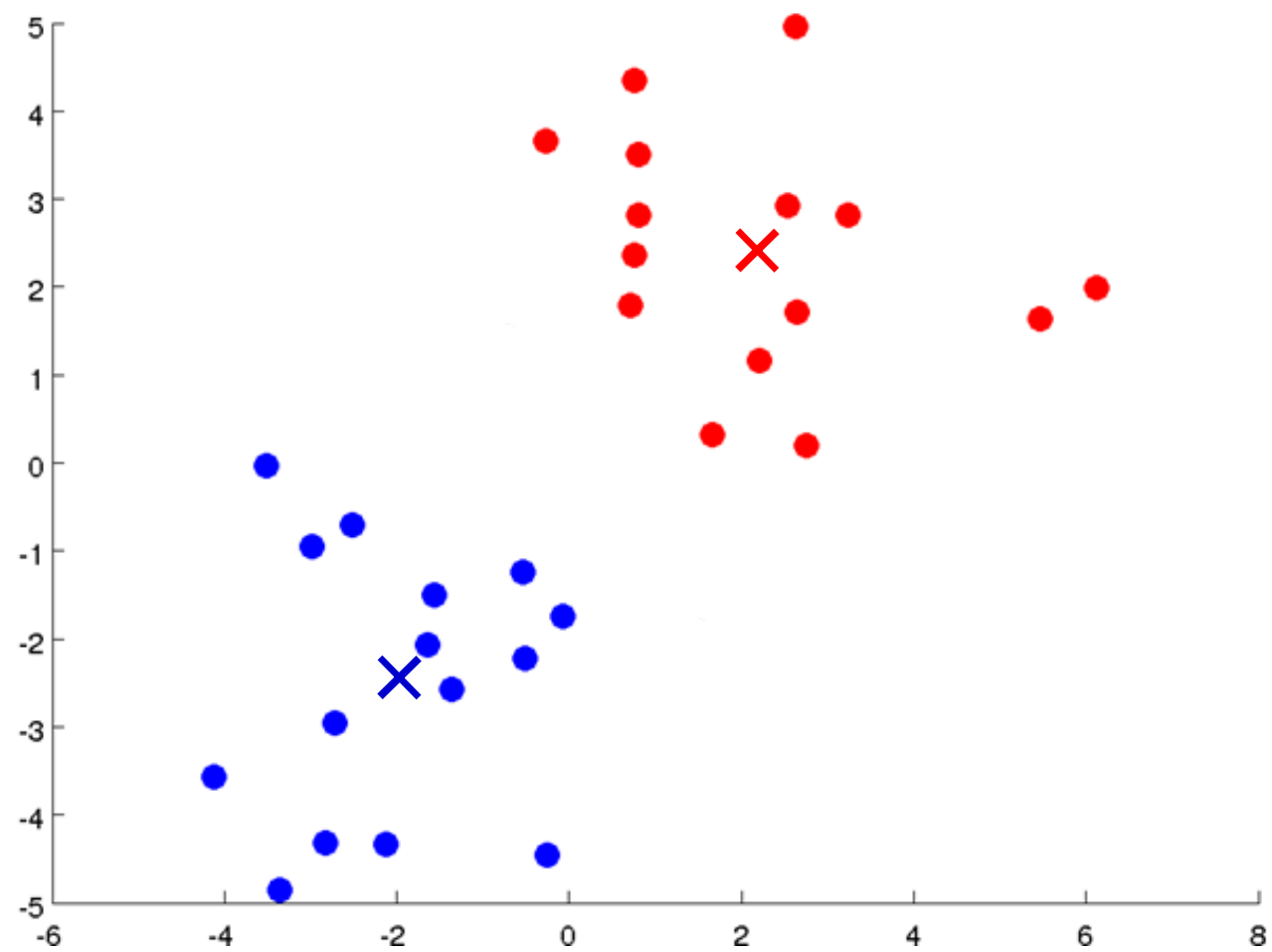












K-means algorithm

Input:

- K (number of clusters)
- Training set $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

$x^{(i)} \in \mathbb{R}^n$ (drop $x_0 = 1$ convention)

K-means algorithm

Randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$

Repeat {

 for $i = 1$ to m

$c^{(i)} :=$ index (from 1 to K) of cluster centroid
 closest to $x^{(i)}$

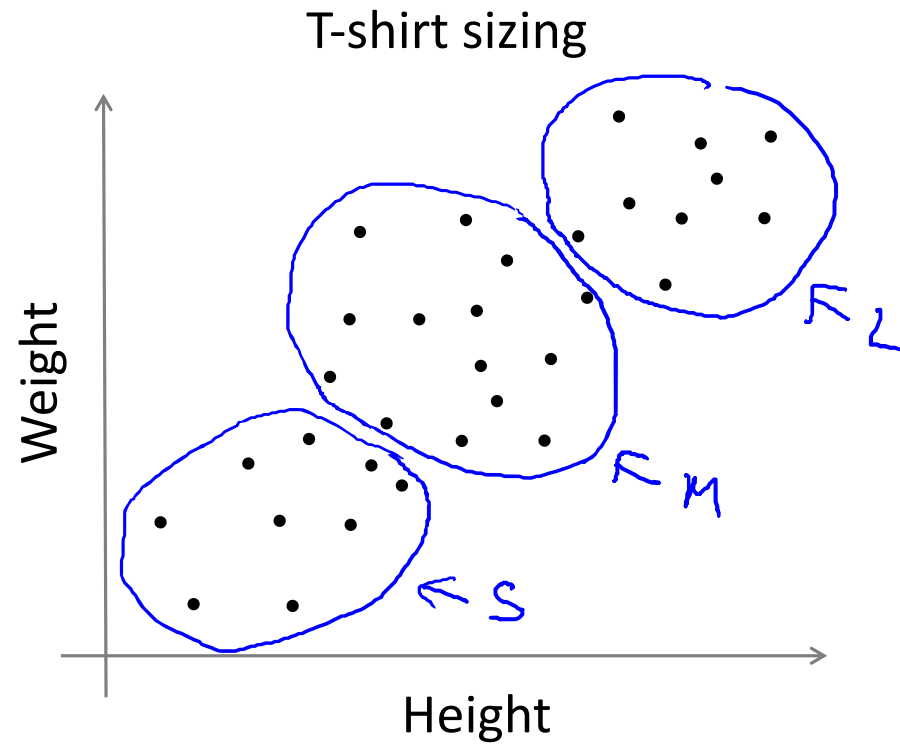
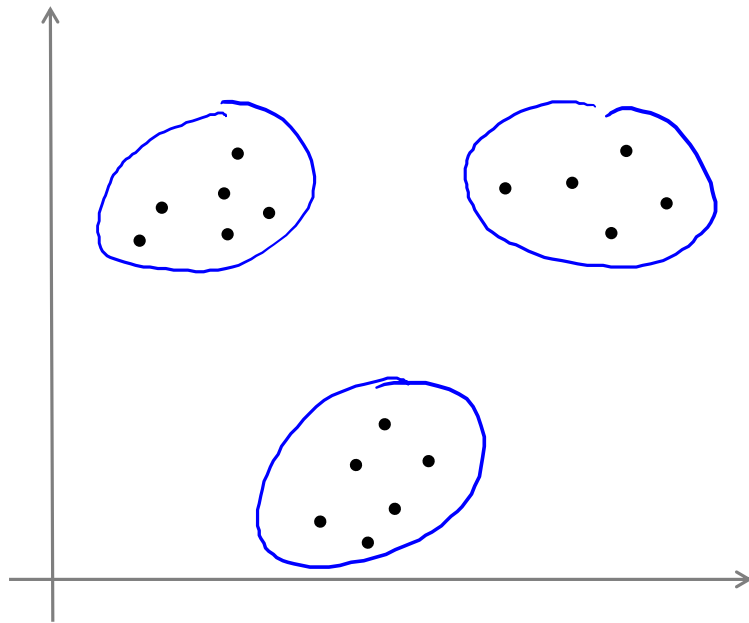
 for $k = 1$ to K

$\mu_k :=$ average (mean) of points assigned to cluster k

}

K-means for non-separated clusters

S, M, L



Clustering

Optimization Objective

K-means optimization objective

$c^{(i)}$ = index of cluster $(1, 2, \dots, K)$ to which example $x^{(i)}$ is currently assigned

μ_k = cluster centroid k ($\mu_k \in \mathbb{R}^n$)

$\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned

Optimization objective:

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m ||x^{(i)} - \mu_{c^{(i)}}||^2$$

$$\min_{\substack{c^{(1)}, \dots, c^{(m)}, \\ \mu_1, \dots, \mu_K}} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$$

K-means algorithm

Randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$

Repeat {

 for $i = 1$ to m

$c^{(i)} :=$ index (from 1 to K) of cluster centroid
 closest to $x^{(i)}$

 for $k = 1$ to K

$\mu_k :=$ average (mean) of points assigned to cluster k

}

Clustering

Random Initialization

K-means algorithm

Randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$

```
Repeat {  
    for  $i = 1$  to  $m$   
         $c^{(i)} :=$  index (from 1 to  $K$ ) of cluster centroid  
            closest to  $x^{(i)}$   
    for  $k = 1$  to  $K$   
         $\mu_k :=$  average (mean) of points assigned to cluster  $k$   
}
```

Random initialization

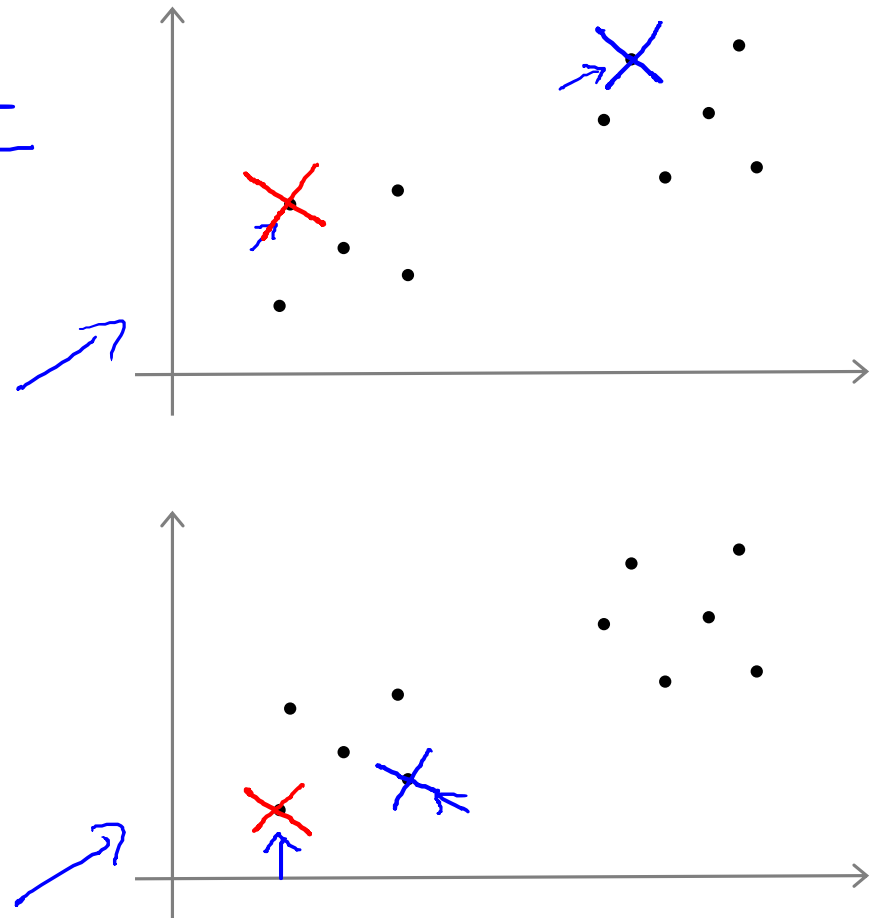
Should have $K < m$

Randomly pick K training examples.

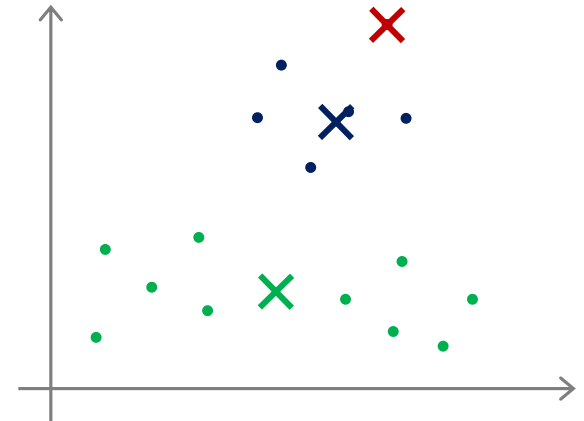
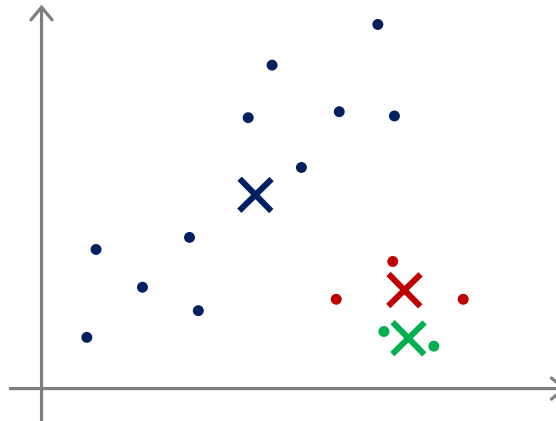
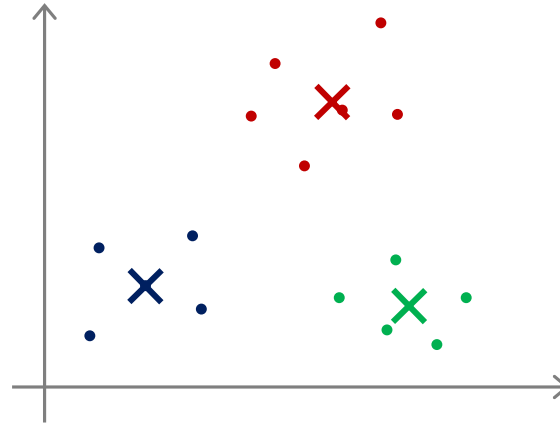
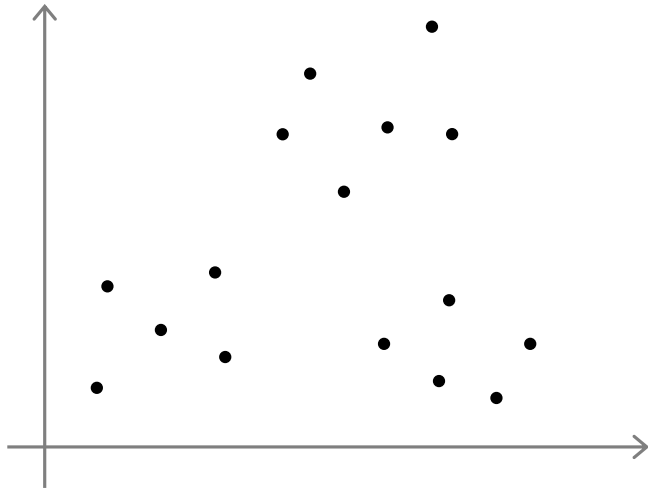
Set $\mu_1, \dots, \mu_K$ equal to these K examples.

$$\begin{aligned}\mu_1 &= x^{(1)} \\ \mu_2 &= x^{(2)} \\ &\vdots\end{aligned}$$

$K=2$



Local optima



Random initialization

For $i = 1$ to 100 {

 Randomly initialize K-means.

 Run K-means. Get $c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K$

 Compute cost function (distortion)

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$$

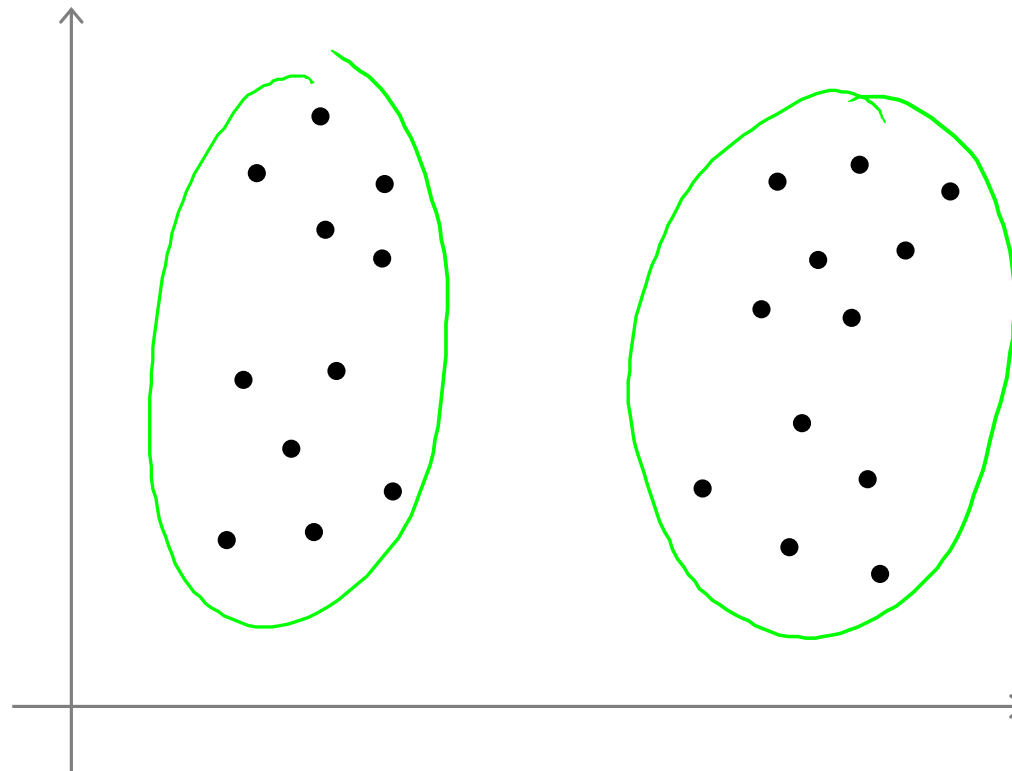
}

Pick clustering that gave lowest cost $J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$

Clustering

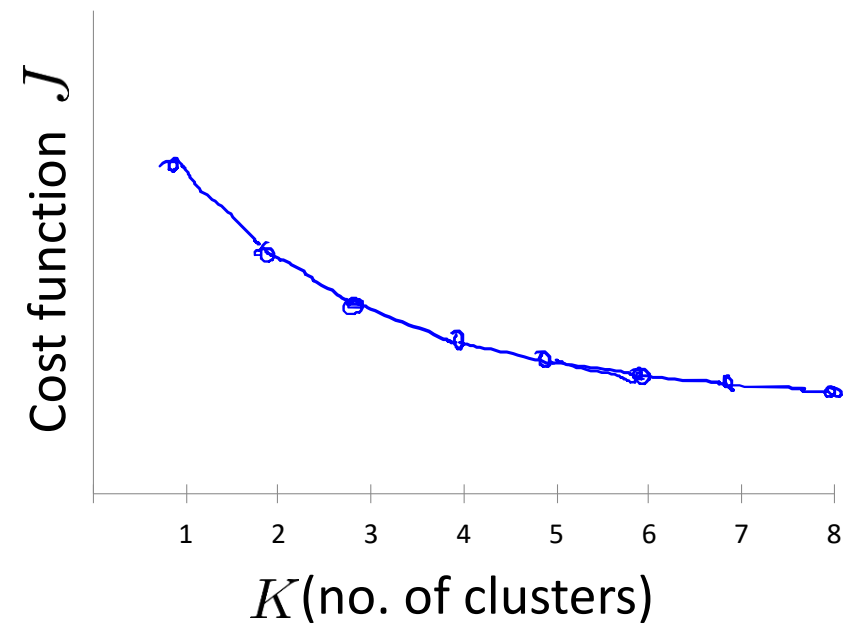
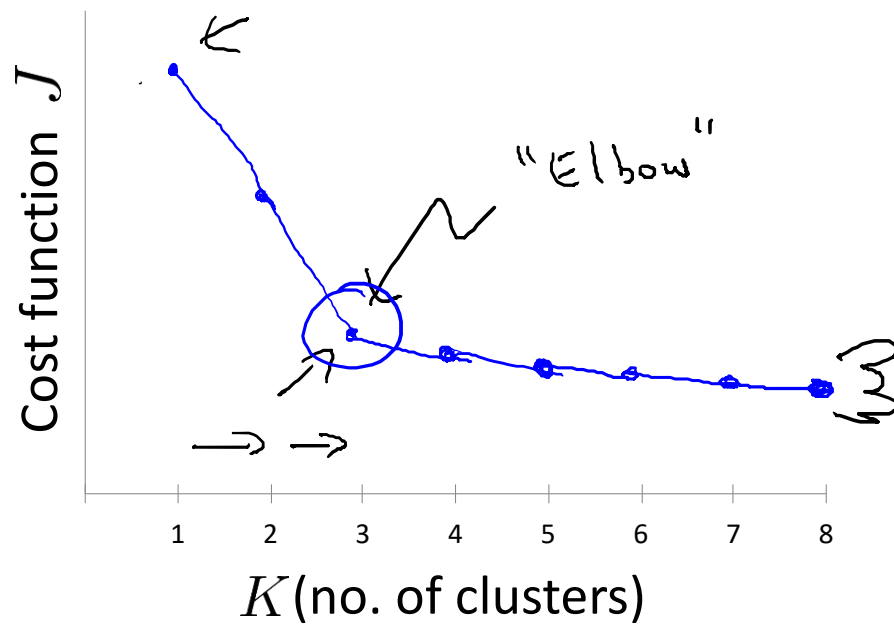
Choosing the number of
clusters

What is the right value of K?



Choosing the value of K

Elbow method:



The Elbow Method works in the following steps:

1. We begin by selecting a range of k values (for example, 1 to 10).
2. For each k , we run K-Means and calculate WCSS (Within-Cluster Sum of Squares), which shows how close the data points are to their cluster centroids.
3. After computing WCSS for all k values, we plot k vs WCSS.
4. WCSS always decreases as k increases because more clusters reduce the internal spread.
5. However, after a certain point, the improvement becomes very small. This bend or “elbow” in the curve indicates the point where adding more clusters no longer gives meaningful improvement.

- **Before the elbow:** WCSS drops quickly -> clusters become much better.

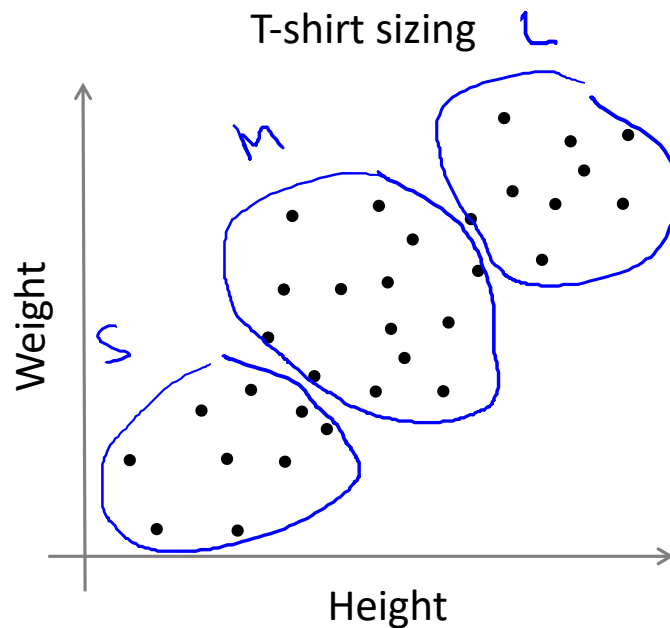
- **After the elbow:** WCSS drops slowly -> extra clusters add little value and may lead to overfitting.

Choosing the value of K

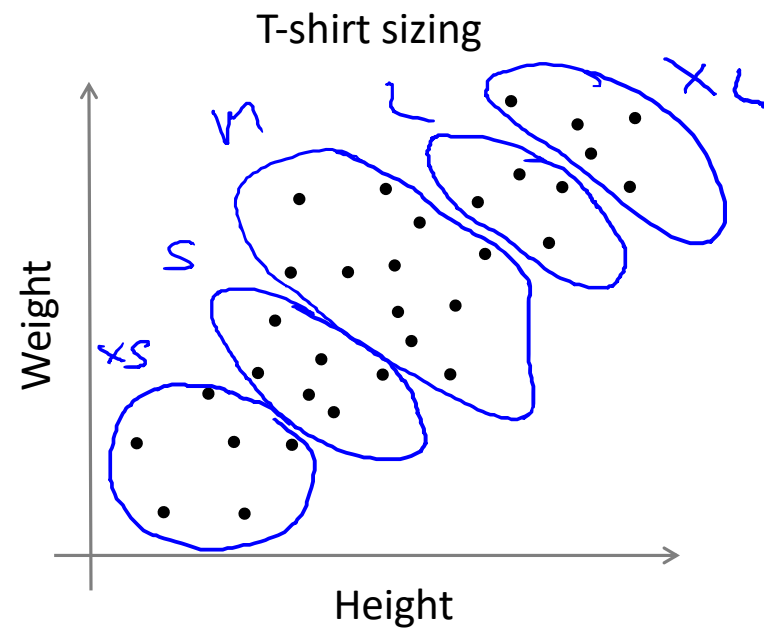
Sometimes, you're running K-means to get clusters to use for some later/downstream purpose. Evaluate K-means based on a metric for how well it performs for that later purpose.

$K=3$ S, M, L

E.g.



$K=5$ XS, S, M, L, XL



Hierarchical Clustering

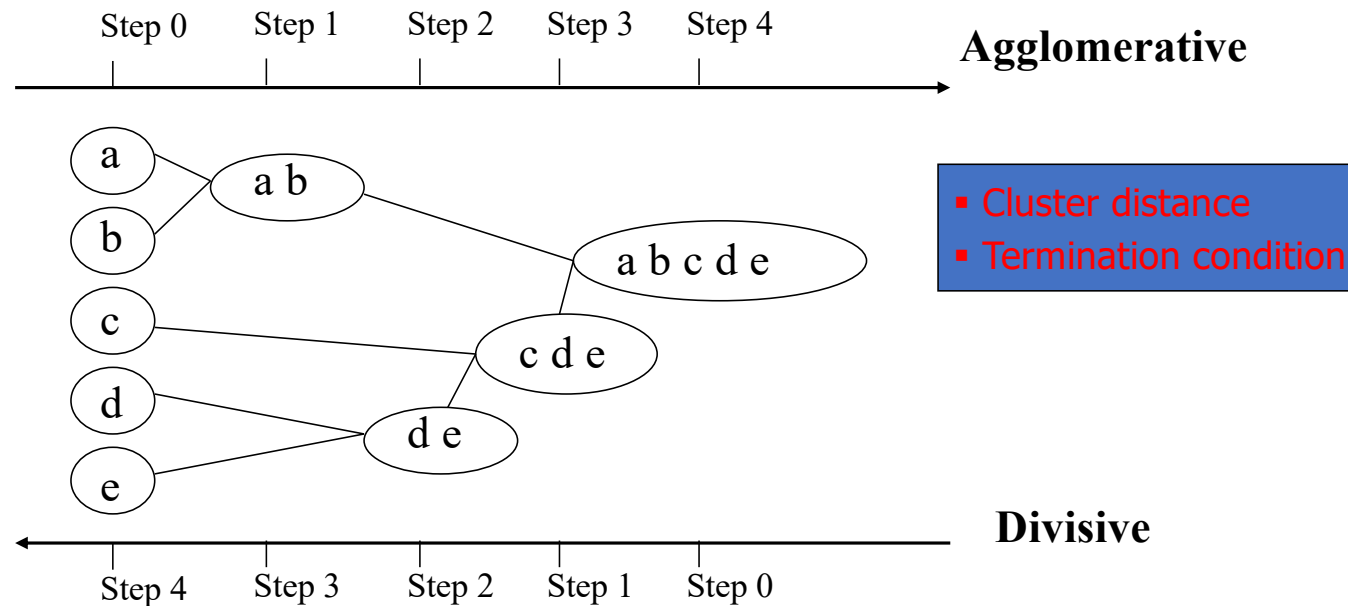
Introduction

- Hierarchical Clustering Approach
 - A typical clustering analysis approach via partitioning data set **sequentially**
 - Construct nested partitions layer by layer via grouping objects into a tree of clusters (**without the need to know the number of clusters in advance**)
 - Use (generalised) distance matrix as clustering criteria
- Agglomerative vs. Divisive
 - **Agglomerative: a bottom-up strategy**
 - Initially each data object is in its own (atomic) cluster
 - Then merge these atomic clusters into larger and larger clusters
 - **Divisive: a top-down strategy**
 - Initially all objects are in one single cluster
 - Then the cluster is subdivided into smaller and smaller clusters
- Clustering Ensemble
 - Using multiple clustering results for robustness and overcoming weaknesses of single clustering algorithms.

Introduction: Illustration

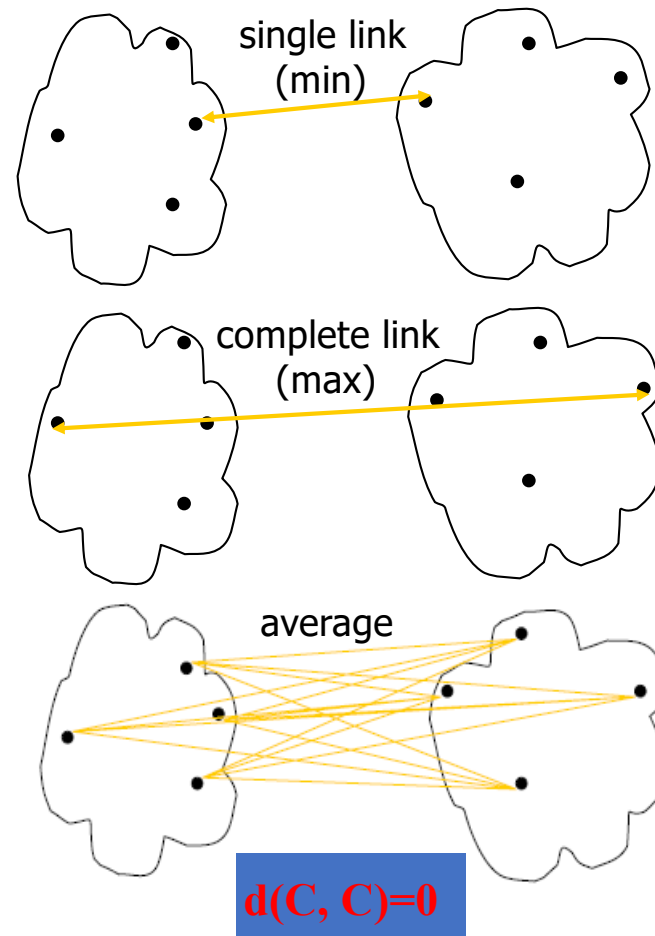
- Illustrative Example: Agglomerative vs. Divisive

Agglomerative and divisive clustering on the data set $\{a, b, c, d, e\}$



Cluster Distance Measures

- **Single link:** smallest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \min\{d(x_{ip}, x_{jq})\}$
- **Complete link:** largest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \max\{d(x_{ip}, x_{jq})\}$
- **Average:** avg distance between elements in one cluster and elements in the other, i.e.,
 $d(C_i, C_j) = \text{avg}\{d(x_{ip}, x_{jq})\}$



Cluster Distance Measures

Example: Given a data set of five objects characterized by a single continuous feature, assume that there are two clusters: $C_1: \{a, b\}$ and $C_2: \{c, d, e\}$.

	a	b	c	d	e
Feature	1	2	4	5	6

1. Calculate the distance matrix .
2. Calculate three cluster distances between C_1 and C_2 .

	a	b	c	d	e
a	0	1	3	4	5
b	1	0	2	3	4
c	3	2	0	1	2
d	4	3	1	0	1
e	5	4	2	1	0

Single link

$$\begin{aligned} \text{dist}(C_1, C_2) &= \min\{d(a, c), d(a, d), d(a, e), d(b, c), d(b, d), d(b, e)\} \\ &= \min\{3, 4, 5, 2, 3, 4\} = 2 \end{aligned}$$

Complete link

$$\begin{aligned} \text{dist}(C_1, C_2) &= \max\{d(a, c), d(a, d), d(a, e), d(b, c), d(b, d), d(b, e)\} \\ &= \max\{3, 4, 5, 2, 3, 4\} = 5 \end{aligned}$$

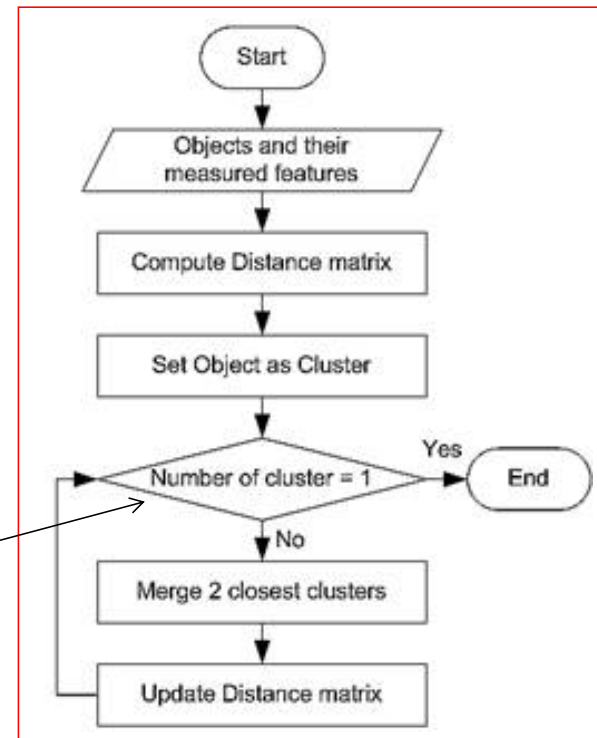
Average

$$\begin{aligned} \text{dist}(C_1, C_2) &= \frac{d(a, c) + d(a, d) + d(a, e) + d(b, c) + d(b, d) + d(b, e)}{6} \\ &= \frac{3 + 4 + 5 + 2 + 3 + 4}{6} = \frac{21}{6} = 3.5 \end{aligned}$$

Agglomerative Algorithm

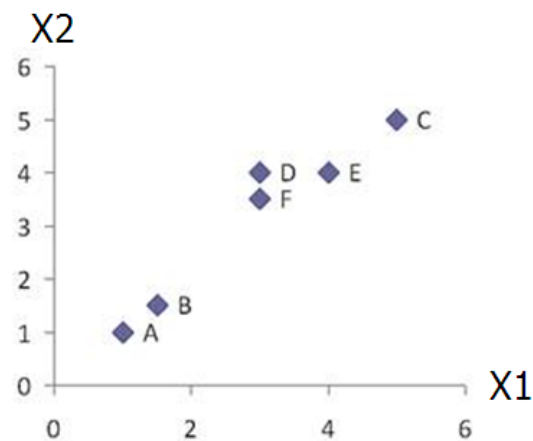
- The *Agglomerative* algorithm is carried out in three steps:

- 1) Convert all object features into a distance matrix
- 2) Set each object as a cluster (thus if we have N objects, we will have N clusters at the beginning)
- 3) Repeat until number of cluster is one (or known # of clusters)
 - Merge two closest clusters
 - Update "distance matrix"



Example

- Problem: clustering analysis with agglomerative algorithm



	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5

data matrix

$$d_{AB} = \left((1-1.5)^2 + (1-1.5)^2 \right)^{\frac{1}{2}} = \sqrt{\frac{1}{2}} = 0.7071$$

$$d_{DF} = \left((3-3)^2 + (4-3.5)^2 \right)^{\frac{1}{2}} = 0.5$$

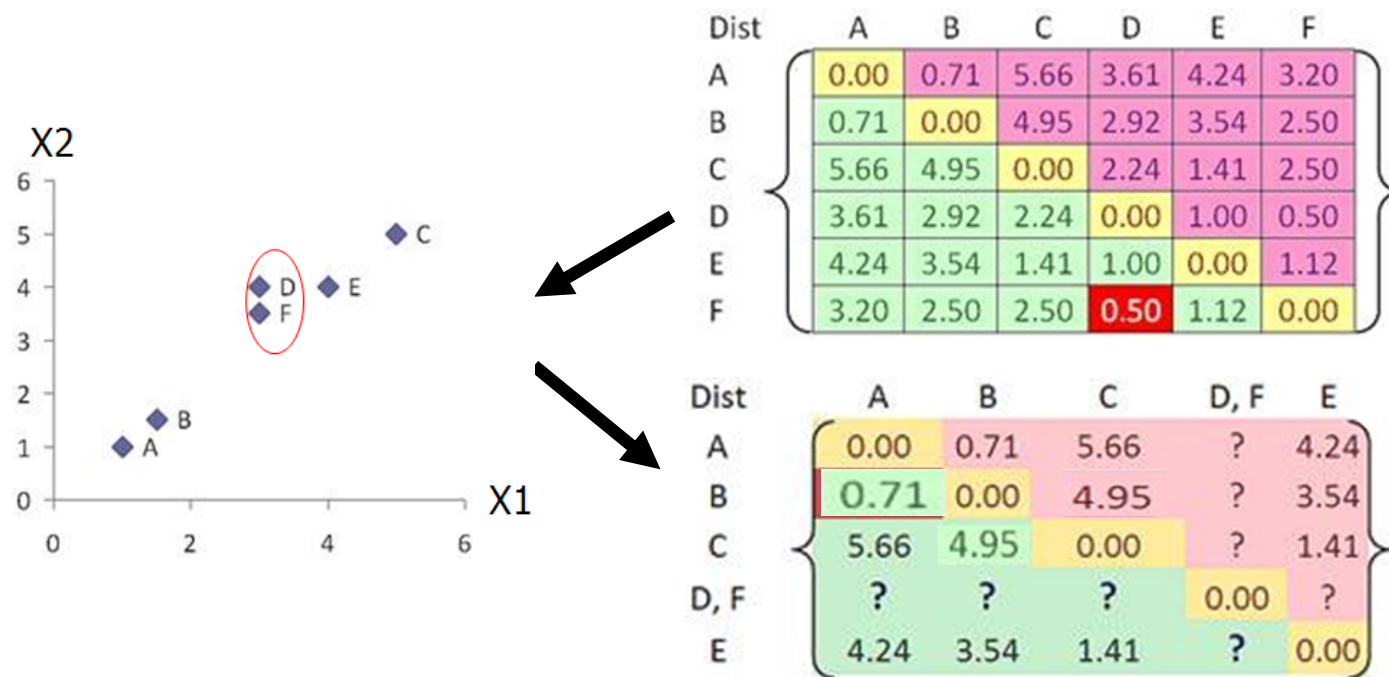
Euclidean distance

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

distance matrix

Example

- Merge two closest clusters (iteration 1)



Example

- Update distance matrix (iteration 1)

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

$$d_{\{D,F\} \rightarrow A} = \min(d_{DA}, d_{FA}) = \min(3.61, 3.20) = 3.20$$

$$d_{\{D,F\} \rightarrow B} = \min(d_{DB}, d_{FB}) = \min(2.92, 2.50) = 2.50$$

$$d_{\{D,F\} \rightarrow C} = \min(d_{DC}, d_{FC}) = \min(2.24, 2.50) = 2.24$$

$$d_{E \rightarrow \{D,F\}} = \min(d_{ED}, d_{EF}) = \min(1.00, 1.12) = 1.00$$

$$d_{\{D,F\} \rightarrow A} = \min(d_{DA}, d_{FA}) = \min(3.61, 3.20) = 3.20$$

$$d_{\{D,F\} \rightarrow B} = \min(d_{DB}, d_{FB}) = \min(2.92, 2.50) = 2.50$$

$$d_{\{D,F\} \rightarrow C} = \min(d_{DC}, d_{FC}) = \min(2.24, 2.50) = 2.24$$

$$d_{E \rightarrow \{D,F\}} = \min(d_{ED}, d_{EF}) = \min(1.00, 1.12) = 1.00$$

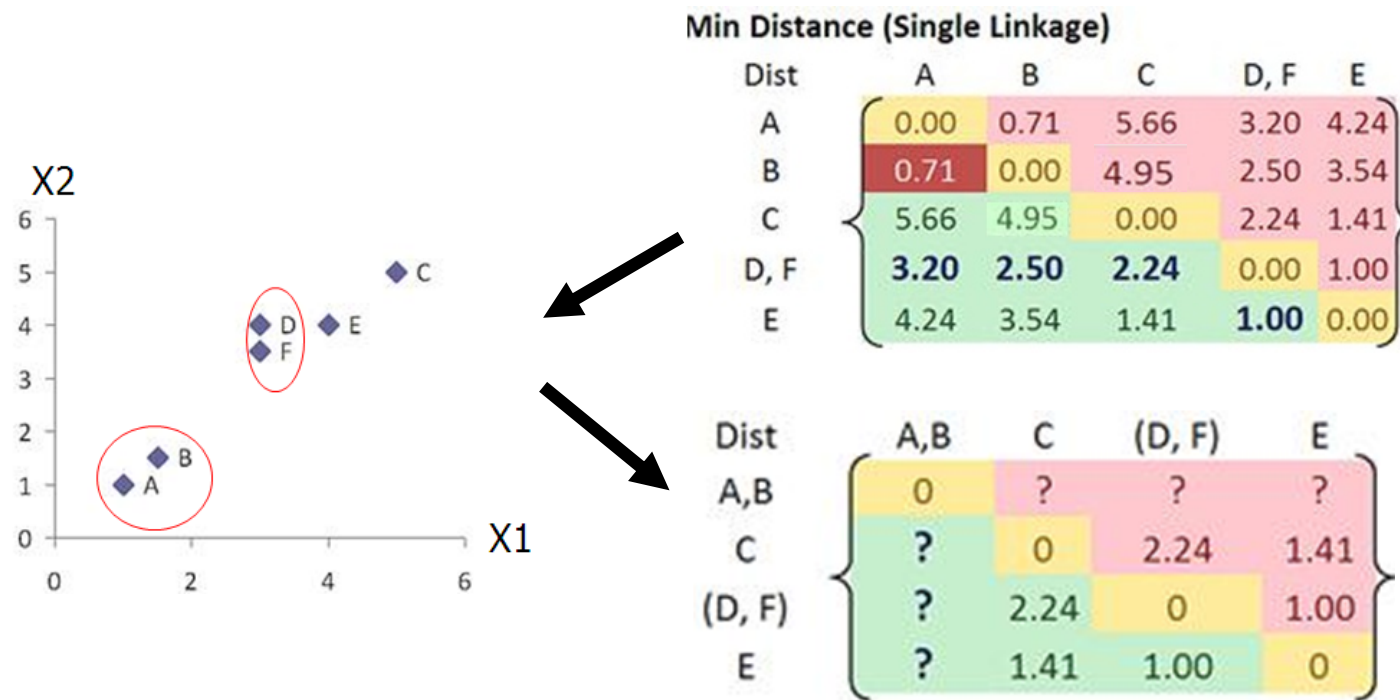
Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

Example

- Merge two closest clusters (iteration 2)



Example

- Update distance matrix (iteration 2)

Min Distance (Single Linkage)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

$d_{C \rightarrow (A,B)} = \min(d_{CA}, d_{CB}) = \min(5.66, 4.95) = 4.95$
 $d_{(D,F) \rightarrow (A,B)} = \min(d_{DA}, d_{DB}, d_{FA}, d_{FB}) = \min(3.61, 2.92, 3.20, 2.50) = 2.50$
 $d_{E \rightarrow (A,B)} = \min(d_{EA}, d_{EB}) = \min(4.24, 3.54) = 3.54$

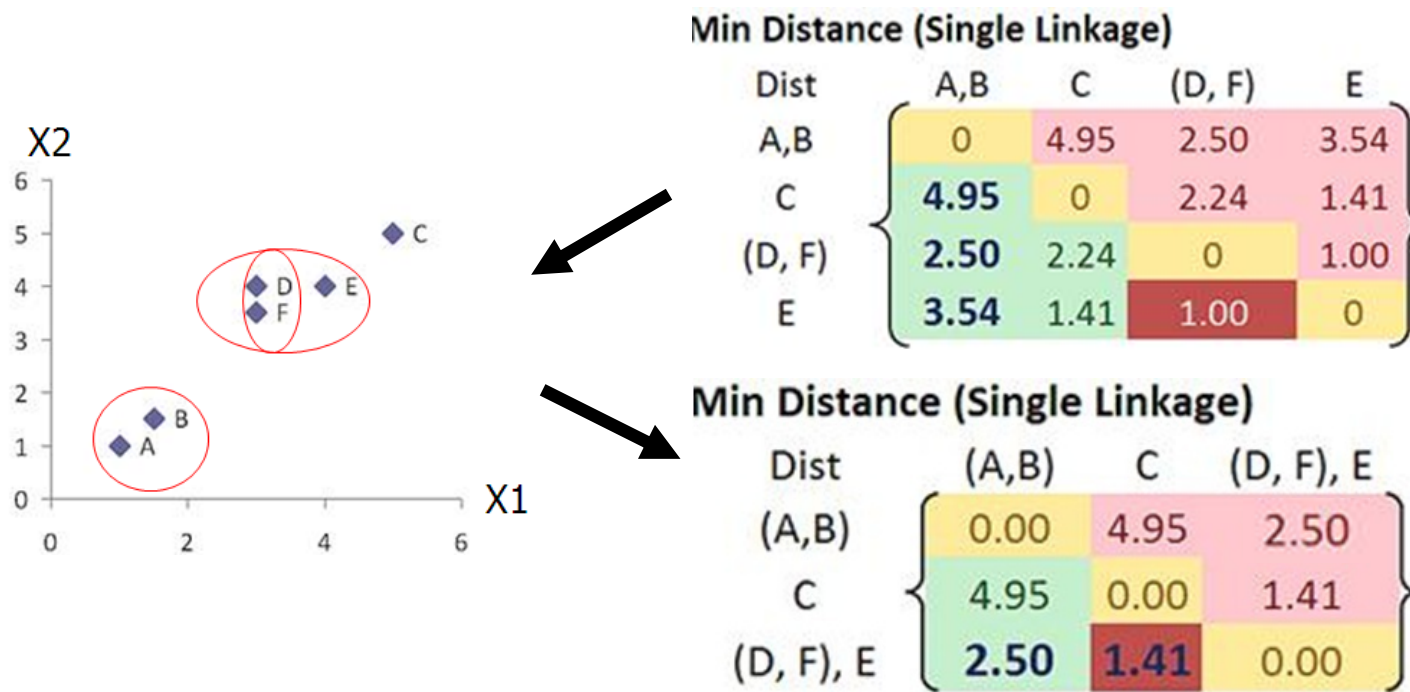
Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

→

Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	4.95	0	2.24	1.41
(D, F)	2.50	2.24	0	1.00
E	3.54	1.41	1.00	0

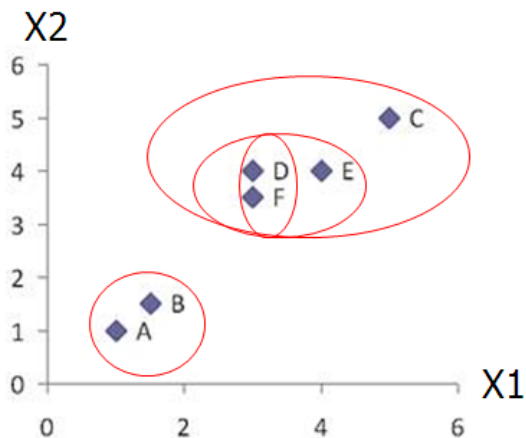
Example

- Merge two closest clusters/update distance matrix (iteration 3)



Example

- Merge two closest clusters/update distance matrix (iteration 4)



Min Distance (Single Linkage)

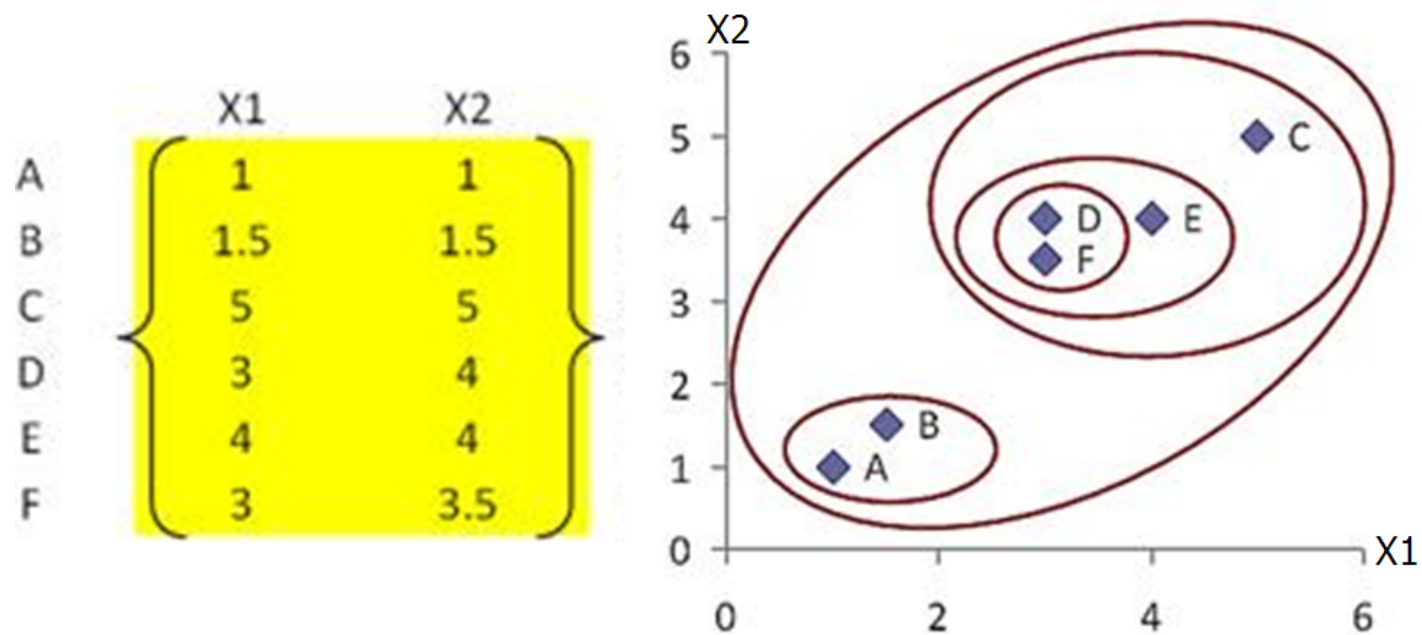
Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	2.50	1.41	0.00

Min Distance (Single Linkage)

Dist	(A,B)	((D, F), E),C
(A,B)	0.00	2.50
((D, F), E),C	2.50	0.00

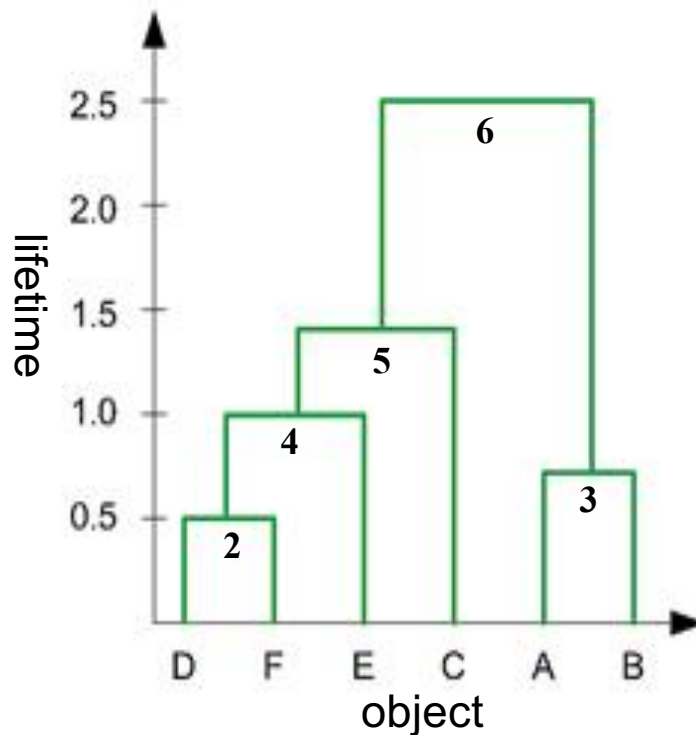
Example

- Final result (meeting termination condition)



Key Concepts in Hierarchical Clustering

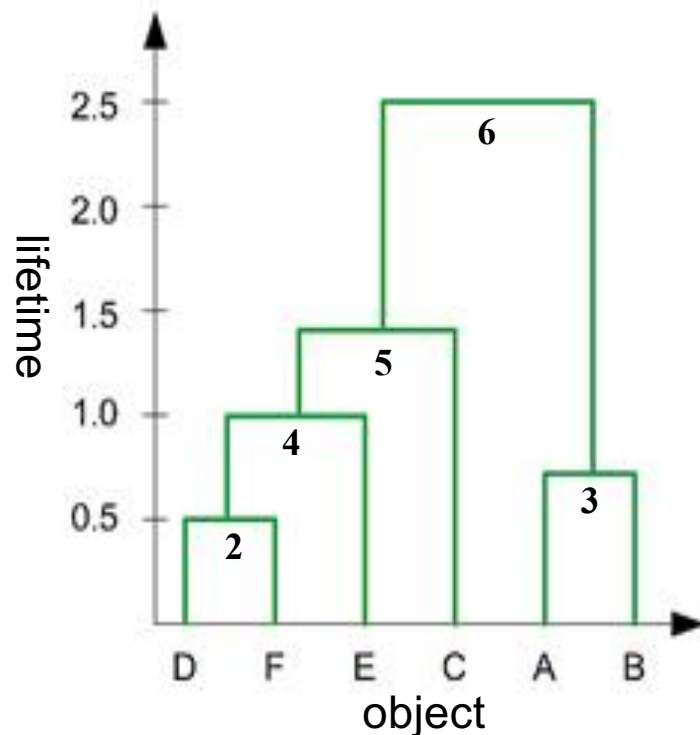
- **Dendrogram tree** representation



1. In the beginning we have 6 clusters: A, B, C, D, E and F
2. We merge clusters D and F into cluster (D, F) at distance 0.50
3. We merge cluster A and cluster B into (A, B) at distance 0.71
4. We merge clusters E and (D, F) into ((D, F), E) at distance 1.00
5. We merge clusters ((D, F), E) and C into (((D, F), E), C) at distance 1.41
6. We merge clusters (((D, F), E), C) and (A, B) into ((((D, F), E), C), (A, B)) at distance 2.50
7. The last cluster contain all the objects, thus conclude the computation

Key Concepts in Hierarchical Clustering

- **Lifetime vs K -cluster Lifetime**



- **Lifetime**

The distance between that a cluster is created and that it disappears (merges with other clusters during clustering).
e.g. lifetime of A, B, C, D, E and F are 0.71, 0.71, 1.41, 0.50, 1.00 and 0.50, respectively, the life time of (A, B) is $2.50 - 0.71 = 1.79$,

- **K -cluster Lifetime**

The distance from that K clusters emerge to that K clusters vanish (due to the reduction to $K-1$ clusters).
e.g.

5-cluster lifetime is $0.71 - 0.50 = 0.21$

4-cluster lifetime is $1.00 - 0.71 = 0.29$

3-cluster lifetime is $1.41 - 1.00 = 0.41$

2-cluster lifetime is $2.50 - 1.41 = 1.09$

Relevant Issues

- How to determine the number of clusters
 - If the number of clusters known, termination condition is given!
 - The ***K*-cluster lifetime** as **the range of threshold value** on the dendrogram tree that leads to the identification of *K* clusters
 - Heuristic rule: **cut a dendrogram tree with maximum life time to find a “proper” *K***
- Major weakness of agglomerative clustering methods
 - Can never undo what was done previously
 - Sensitive to cluster distance measures and noise/outliers
 - Less efficient: $O(n^2 \log n)$, where *n* is the number of total objects
- There are several **variants** to overcome its weaknesses
 - **BIRCH**: scalable to a large data set
 - **ROCK**: clustering categorical data
 - **CHAMELEON**: hierarchical clustering using dynamic modelling

Summary

- **Hierarchical** algorithm is a sequential clustering algorithm
 - Use distance matrix to construct a tree of clusters (**dendrogram**)
 - Hierarchical representation without the need of knowing # of clusters (can set termination condition with known # of clusters)
- Major weakness of agglomerative clustering methods
 - Can never undo what was done previously
 - Sensitive to cluster distance measures and noise/outliers
 - Less efficient: $O(n^2 \log n)$, where n is the number of total objects